

COMPUTACIÓN DE ALTAS PRESTACIONES PRÁCTICA 1

AUTORES

CARLOS GARCÍA SANTA
CELIA MARTÍN ÁLVAREZ

GRUPO 1461 : PAREJA 03

INGENIERÍA INFORMÁTICA
ESCUELA POLITÉCNICA SUPERIOR
UNIVERSIDAD AUTÓNOMA DE MADRID



07/10/2024

Índice

Índice	2
EJERCICIO 1: VECTORIZACIÓN AUTOMÁTICA CON EL COMPILADOR	3
EJERCICIO 2: USO DE INTRINSICS PARA VECTORIZAR MANUALMENTE	8
EJERCICIO 4: VECTORIZAR UN ALGORITMO DE PROCESAMIENTO DE IMAGEN	11

EJERCICIO 1: VECTORIZACIÓN AUTOMÁTICA CON EL COMPILADOR

1.1. Realice los siguientes pasos y documente en la memoria los resultados obtenidos:

Identifique las instrucciones SIMD admitidas por el microprocesador de la computadora del laboratorio (o su computadora). Indique el modelo de CPU y las instrucciones SIMD que admite y la versión del compilador utilizado.

```
celia@celia-virtual-machine:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          45 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 2
On-line CPU(s) list:    0,1
Vendor ID:              GenuineIntel
Model name:             11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz
CPU family:             6
Model:                  140
Thread(s) per core:     1
Core(s) per socket:     1
Socket(s):              2
Stepping:               1
BogoMIPS:               5606.40
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge m
ca cmov pat pse36 clflush mmx fxsr sse sse2 ss syscall
nx pdpe1gb rdtscp lm constant_tsc arch_perfmon rep_go
od nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_k
nown_freq pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse
4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave a
vx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch cp
uid_fault ssbd ibrs ibpb stibp ibrs_enhanced fsgsbase
tsc_adjust bmi1 avx2 smep bmi2 erms invpcid avx512f av
x512dq rdseed adx smap avx512ifma clflushopt clwb avx5
12cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1
xsaves arat avx512vbmi umip pku ospke avx512_vbmi2 gfn
i vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpo
pcntdq rdpid movdiri movdir64b fsrm avx512_vp2intersec
t md_clear flush_l1d arch_capabilities
```

Figura 1: Información de cpu (lscpu)

```
[santacg@archlinux-santacg ~]$ gcc --version
gcc (GCC) 14.2.1 20240910
Copyright (C) 2024 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Figura 2: Versión del compilador

Al ejecutar **lscpu** en la terminal se muestran todas las instrucciones que nuestro procesador soporta. Podemos observar las diferentes arquitecturas de instrucciones **SIMD** estando **MMX**, **SSE** y **AVX** incluidas. Tenemos un modelo de procesador **Intel Core i7-1165G7** de onceava generación y una versión del compilador **gcc 14.2.1**.

Partiendo del código de ejemplo **simple2.c**, revise el código y compruebe que es un programa simple que realiza una multiplicación-suma básica de matrices muchas veces dentro de un bucle.

```
/* Perform an operation a number of times */
for (t=0; t < NUMBER_OF_TRIALS; t++) {
    for (i=0; i < ARRAY_SIZE; i++) {
        c += m*a[i] + b[i];
    }
}
```

Figura 2: Bucle donde se realizan muchas operaciones sobre un elemento

En esta parte del código podemos ver un bucle que se ejecuta un número de intentos determinado por la macro **NUMBER_OF_TRIALS**, dentro de este se realiza otro bucle donde se opera aplicando una multiplicación entre el *i*-ésimo elemento del array “**a**” por una constante “**m**” y esto se suma al *i*-ésimo elemento del array “**b**”, esta operación se acumula en la variable “**c**”.

El compilador GCC aplicará automáticamente la vectorización en el nivel de optimización **-O3**. Pruebe el nivel **-O3** e inspeccione el informe de vectorización para verificar si se ha vectorizado algún bucle.

```
celia@celia-virtual-machine:~/Downloads/material$ gcc -O3 -march=native -fopt-info-vec-optimized -o simple2 simple2.c
simple2.c:26:21: optimized: loop vectorized using 32 byte vectors
simple2.c:19:17: optimized: loop vectorized using 32 byte vectors
```

Figura 3: Bucles vectorizados

Al ejecutar el comando podemos ver que el bucle que puebla los arrays y el bucle interno que realiza la operación descrita anteriormente han sido vectorizados por nuestro compilador.

```
celia@celia-virtual-machine:~/Downloads/material$ gcc -O3 -march=native -fno-tree-vectorize -fopt-info-vec-optimized -o simple2_no_vec simple2.c
celia@celia-virtual-machine:~/Downloads/material$
```

Figura 4: Ejecución simple2.c

Podemos ver que al ejecutar el comando **gcc -O3 -march=native -fno-tree-vectorize -fopt-info-vecoptimized -o simple2_no_vec simple2.c** no muestra nada, como indica en el enunciado.

Explicar las principales diferencias entre los códigos ensambladores centrándonos en las instrucciones SIMD generadas por el compilador y los registros SIMD.

En **simple2_o3** se vectoriza utilizando registros **xmm** junto con instrucciones vectoriales como **movdqa**, **paddb** y **movaps**, que operan en varios datos a la vez.

En cambio, en **simple2_o3_native**, aunque también se empleen registros **xmm**,

todas las instrucciones son escalares como **movsd**, **mulsd** y **addsd**, sin aprovechar la paralelización ofrecida por la vectorización, la razón por la que se usan estos registros a pesar de no vectorizar el código es por las arquitecturas y compiladores modernos que los utilizan para realizar operaciones de punto **flotante**.

1.2. Compare el rendimiento de un código compilado con diferentes opciones de compilación en los flag que afectan el rendimiento, al menos pruebe con las combinaciones más significativas de los flag: -O3, -O2, -O1, con y sin vectorización -fno-tree-vectorize.

La aceleración se calcula dividiendo el tiempo normal sin optimización entre el tiempo optimizado.

Optimización	No vectorizado (tiempo en segundos)	Vectorizado (tiempo en segundos)	Aceleración vectorizado
-O3	8,39	8,25	1,017
-O2	8,49	8,36	1,016
-O1	8,72	8,47	1,030

Como podemos observar el tiempo de ejecución del código no vectorizado es superior al tiempo de ejecución del vectorizado, en cualquier nivel de optimización. Esto se debe a que cuando se vectoriza el código este empieza a realizar las operaciones de forma paralela mientras que con el no vectorizado se hace de forma secuencial. También observamos que cuanto mayor es el grado de optimización menor es el tiempo de ejecución, ya que con los niveles sucesivos se aplican más optimizaciones que mejoran el tiempo de ejecución.

1.3. El compilador con el flag -march permite crear código para arquitecturas diferentes de la nativa, máquina en la que se ejecuta el

compilador(-march=native). Esto nos permite ver cómo sería el código generado en arquitecturas muy diferentes, aunque no lo podamos ejecutar, este proceso se denomina crosscompilación.

```
19.650671celia@celia-virtual-machine:~/Downloads/mgcc -O3 -march=xxx simple2.c
cc1: error: bad value ('xxx') for '-march=' switch
cc1: note: valid arguments to '-march=' switch are: nocona core2 nehalem corei7
westmere sandybridge corei7-avx ivybridge core-avx-i haswell core-avx2 broadwell
skylake skylake-avx512 cannonlake icelake-client rocketlake icelake-server casc
adelake tigerlake cooperlake sapphirerapids alderlake bonnell atom silvermont sl
m goldmont goldmont-plus tremont knl knm x86-64 x86-64-v2 x86-64-v3 x86-64-v4 ed
en-x2 nano nano-1000 nano-2000 nano-3000 nano-x2 eden-x4 nano-x4 k8 k8-sse3 opte
ron opteron-sse3 athlon64 athlon64-sse3 athlon-fx amdfam10 barcelona bdver1 bdve
r2 bdver3 bdver4 znver1 znver2 znver3 btver1 btver2 native
```

Figura 5: Posibles etiquetas para march

Al poner el comando con “xxx” junto al **march** nos salen todas las posibles etiquetas que le pueden acompañar. Hemos usado para comparar códigos las etiquetas **athlon64** para la arquitectura **SSE**, **tigerlake** para la arquitectura **AVX** y **zven5** para la arquitectura **AVX512**.

En primer lugar, podemos observar que en **SSE** el programa usa un mayor número de instrucciones, **~94 instrucciones** en total tras compilar simple2.c, frente a **~50** de **AVX** y **~56** de **AVX512**, esto claramente se debe al menor tamaño de los registros de la arquitectura **SSE** que son los **xmm** de **128** bits frente a los **ymm** de **256** bits y **zmm** de **512** bits de **AVX** y **AVX512** respectivamente, dado este menor tamaño se requiere un mayor número de instrucciones para ejecutar el mismo código. Por otro lado, tanto en **AVX** como en **AVX512** se emplean registros **xmm**, estos se refieren a la parte baja de 128 bits de los registros **ymm** en **AVX** o **zmm** en **AVX512**, en esta última adicionalmente también se emplean los **ymm** de **AVX** pero para referenciar la parte baja de 256 bits de los **zmm**.

Atendiendo a las instrucciones se puede notar una clara diferencia entre **SSE** y las **AVX**, teniendo las instrucciones de la primera 2 operandos, frente a los 3 operandos que emplean muchas de las instrucciones **AVX**, esto denota una mayor complejidad y paralelización, puesto que se operan sobre registros de mayor tamaño y con un mayor número de registros por cada instrucción.

EJERCICIO 2: USO DE INTRINSIC PARA VECTORIZAR

MANUALMENTE

Para vectorizar el segundo bucle se ha seguido el tutorial proporcionado en el enunciado de la práctica correspondiente al ejercicio.

2.1 Realice la vectorización del primer bucle (de inicialización) utilizando intrínsecos. Consejo: cree vectores iniciales.

```
__m256d vb = {0.0, 1.0, 2.0, 3.0};
__m256d va = {1.0, 2.0, 3.0, 4.0};

__m256d constant = {4.0, 4.0, 4.0, 4.0};
/* Populate A and B arrays */
for (i = 0; i < ARRAY_SIZE; i += 4) {
    _mm256_store_pd(p: &b[i], a: vb);
    _mm256_store_pd(p: &a[i], a: va);
    vb = _mm256_add_pd(a: vb, b: constant);
    va = _mm256_add_pd(a: va, b: constant);
}
```

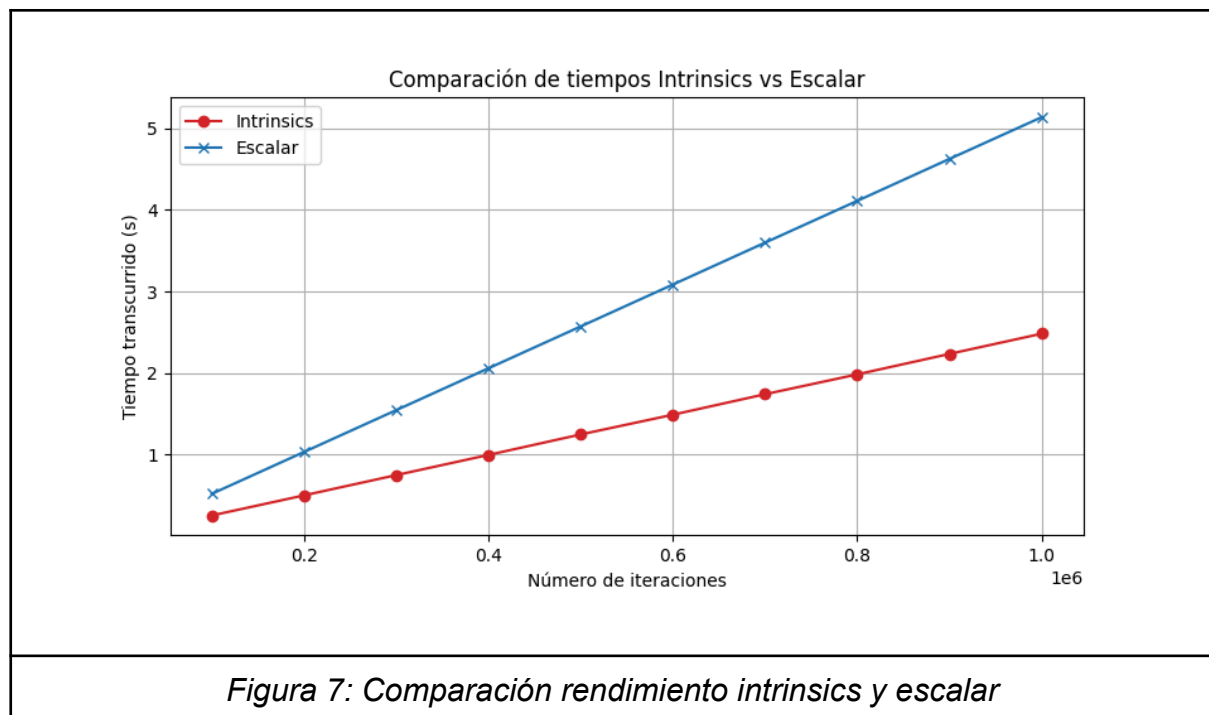
Figura 6: Código vectorizado con intrínsecos del primer bucle

Primero se inicializan dos variables del tipo de datos **__m256d** que representa un vector capaz de almacenar **4 doubles** ($2^8 - 2^6 = 2^2$) y una constante del mismo tipo de datos también inicializada. En el bucle se hace la asignación de los 4 valores de

los vectores “va” y “vb” a los **256 i-ésimos bits** de los arrays “a” y “b”

respectivamente con la instrucción `_mm256_store_pd`, posteriormente se realiza la operación de suma de vectores con la instrucción `_mm256_add_pd` que realiza la suma entre dos vectores de **4 packet doubles** cada uno, sumando en concreto, los vectores “va” y “vb” con la constante y asignando este resultado a los propios vectores, de esta manera el bucle continúa iterando hasta poblar los arrays de vectores.

2.2 Obtenga el tiempo de ejecución para diferentes valores de NUMBER_OF_TRIALS: de 100.000 a 1.000.000 en pasos de 100.000. Dibuje el resultado en un gráfico que compare los resultados con y sin vectorización. Explique los resultados.



En el gráfico, se observa claramente la mejora en el tiempo de ejecución proporcionada por el uso de **vectorización** a través de **intrinsics**. Esta mejora es evidente desde los primeros valores del número de pruebas (variable “**number of**

trials”), además, la tendencia muestra que conforme se incrementa la variable la diferencia en el rendimiento entre las versiones con y sin vectorización se mantiene proporcionalmente.

2.3 Calcule la aceleración (Tiempo no vectorizado/ Tiempo vectorizado) obtenida y represéntela gráficamente.

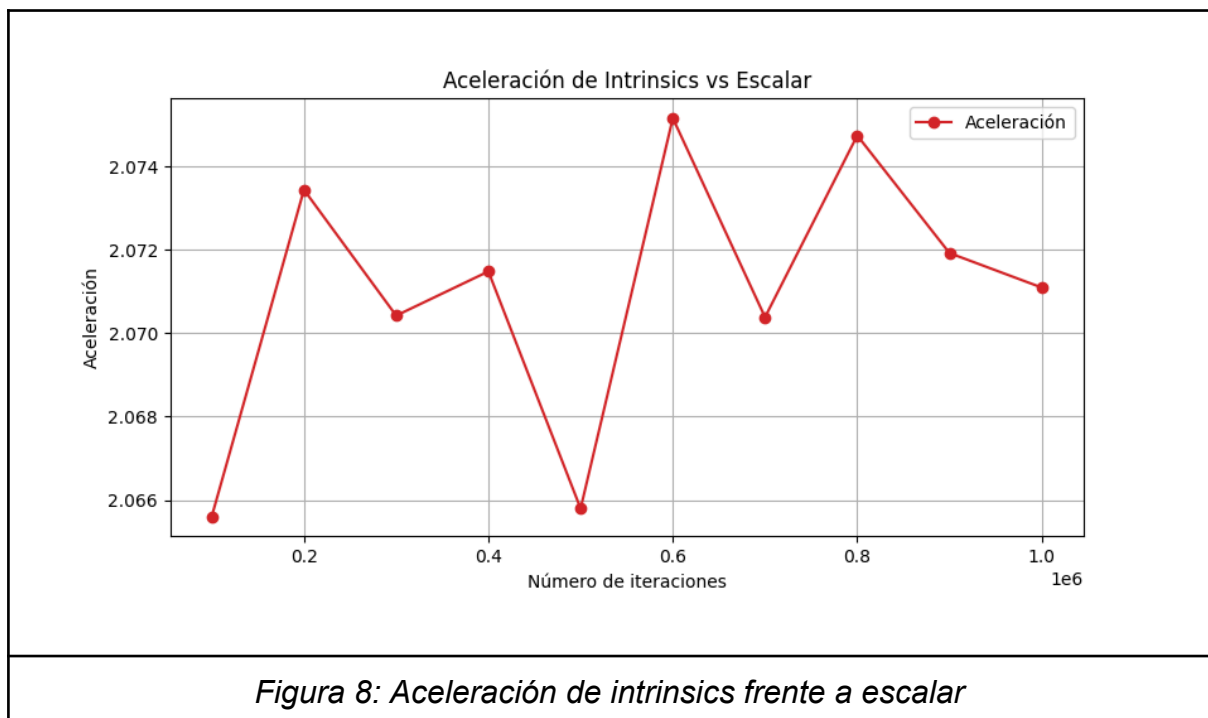


Figura 8: Aceleración de intrinsic frente a escalar

La aceleración se mantiene en un rango entre 2.07 y 2.08, esto parece indicarnos que el **factor de aceleración** en este problema concreto se mantiene fijo, obteniéndose aproximadamente un doble de mejora en el tiempo de ejecución en función del tamaño del problema. La mejora es significativa con lo cual se ha logrado **paralelizar** el problema de forma satisfactoria, sin embargo, dista de la mejora teórica ideal que equivaldría a un factor de aceleración igual al tamaño del vector en este caso 4.

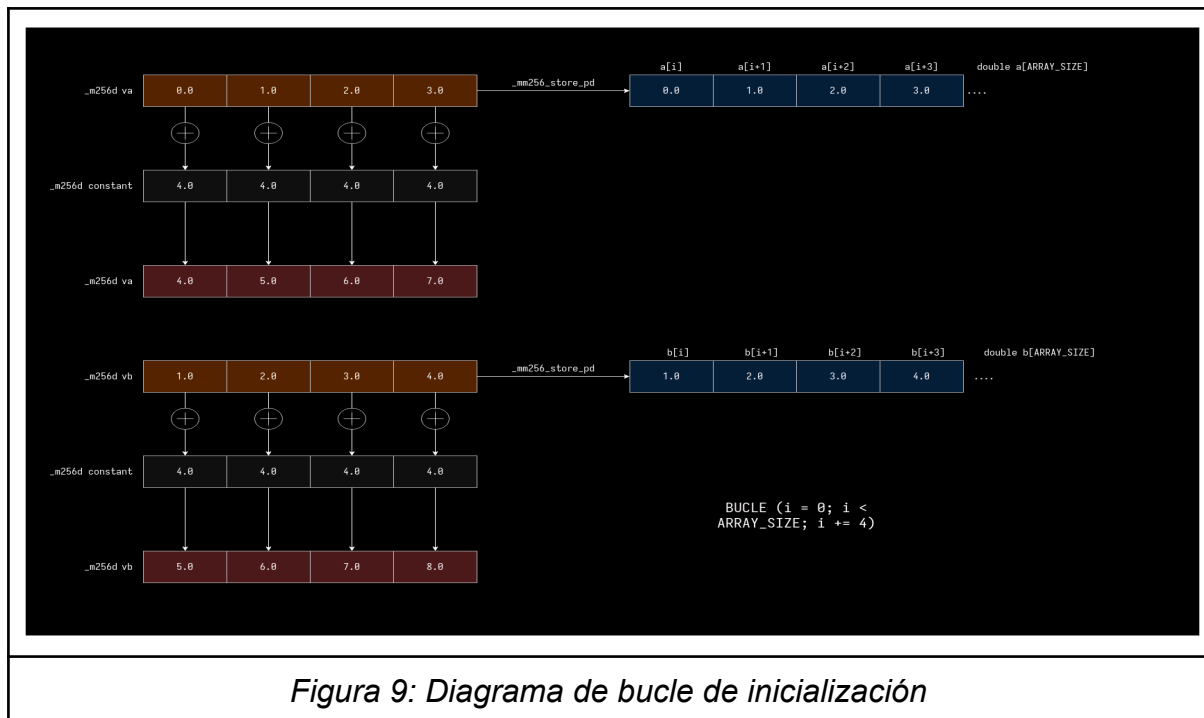


Figura 9: Diagrama de bucle de inicialización

EJERCICIO 4: VECTORIZAR UN ALGORITMO DE PROCESAMIENTO DE IMAGEN

El programa itera sobre los píxeles de una imagen o varias imágenes y obtiene sus valores RGB para después transformar la imagen a escala de grises, multiplicando cada pixel por una constante. Se compila con la opción `-O3` y al ejecutarse muestra la anchura, altura y número de canales de la imagen, y el tiempo de ejecución del bucle de transformación a gris.

El segundo bucle (Bucle 1) que contiene el algoritmo de la escala de grises es óptimo para la **vectorización**, puesto que consiste en una multiplicación iterativa de **múltiples datos** (en este caso los valores `r`, `g` y `b` de la imagen) por una constante para la transformación a grises. Si utilizamos `-O3` para compilar y activamos el reporte de auto vectorización (`-fopt-info-vec-optimized`) se observa como este bucle es vectorizado.

```
greyScale.c:74:25: optimized: loop vectorized using 32 byte vectors
greyScale.c:74:25: optimized: loop versioned for vectorization because of possible aliasing
greyScale.c:74:25: optimized: loop vectorized using 16 byte vectors
```

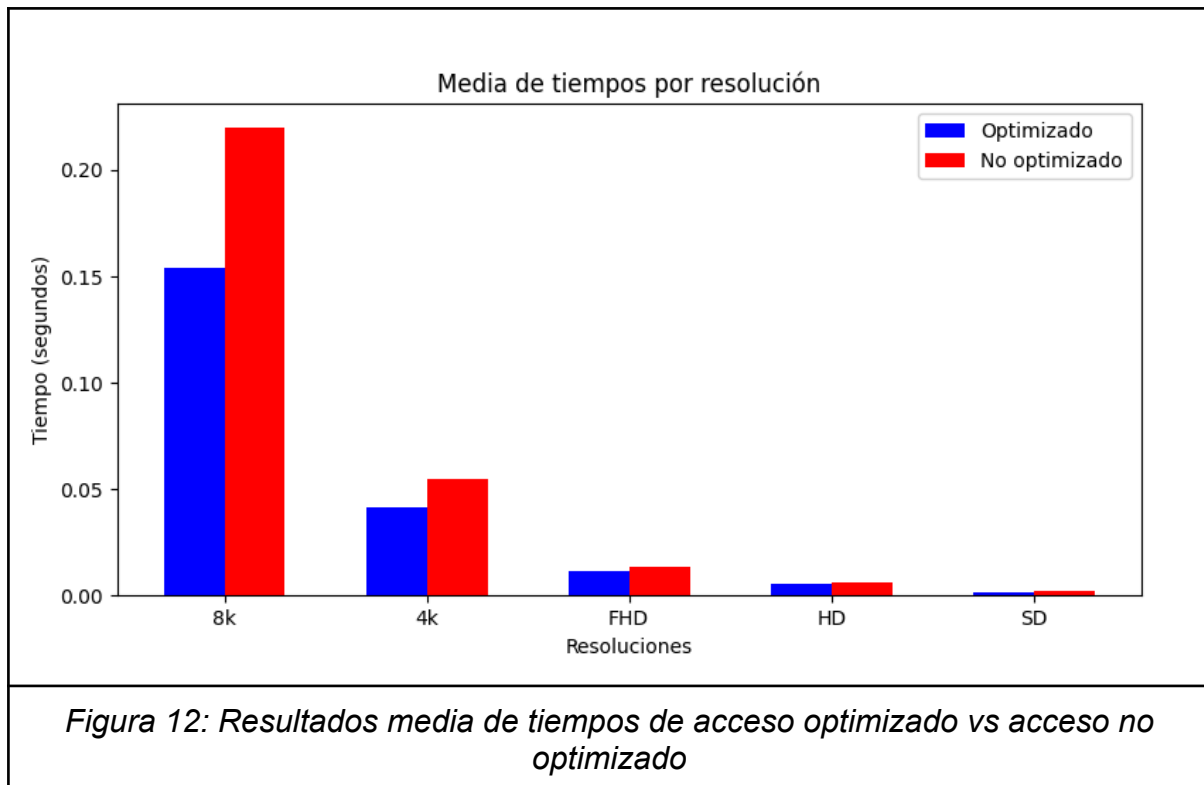
Figura 10: Resultados de compilación de greyScale con -fopt-info-vec-optimized

En este bucle además se puede optimizar el acceso a memoria, accediendo de tal forma que se recorra la imagen en el mismo orden en el que se almacena en memoria, es decir, primero a lo ancho y luego a lo largo.

Ejecutando 10 veces el programa con cada imagen obtenemos una media de tiempo de ejecución de **0.05907 ~ 0.06** segundos de media para el bucle sin optimizar el acceso, frente a **0.04381 ~ 0.04** segundo de media del bucle con acceso a memoria optimizado. Además la diferencia es bastante notable para imágenes grandes 4k u 8k donde vemos diferencias de 10 décimas de segundo. Dada esta mejora esta optimización se utilizará para las posteriores modificaciones de código.

```
// RGB to grey scale
int r, g, b;
for (int i = 0; i < height; i++) {
    for (int j = 0; j < width; j++) {
        getRGB(im: rgb_image, width, height, nchannels: 4, x: j, y: i, r: &r, g: &g, b: &b);
        grey_image[i * width + j] = (int)(0.2989 * r + 0.5870 * g + 0.1140 * b);
    }
}
```

Figura 11: Bucle de escala de grises con acceso optimizado



```

for (int i = 0; i < width; i += 4)
{
    __m256i orden = _mm256_setr_epi32(0, 4, 1, 5, 0, 2, 0, 1);
    __m256 coeficiente = _mm256_setr_ps(0.2989, 0.5870, 0.1140, 0.0, 0.2989, 0.5870, 0.1140, 0.0); //coeficientes r, g y b
    // getRGB(rgb_image, width, height, 4, i, j, &r, &g, &b);
    //cargamos en los vectores el pixel
    __m128i vec1 = _mm_loadl_epi64((__m128i *) (rgb_image + (i + width * j) * 4));
    __m128i vec2 = _mm_loadl_epi64((__m128i *) (rgb_image + (i + width * j) * 4 + 8));

    //Hay que convertir los vectores en float para poder operar con ellos primero lo convertimos
    //entero y luego a float
    __m256i vec1i = _mm256_cvtepu8_epi32(vec1);
    __m256i vec2i = _mm256_cvtepu8_epi32(vec2);

    __m256 vec1f = _mm256_cvtepi32_ps(vec1i);
    __m256 vec2f = _mm256_cvtepi32_ps(vec2i);

    // Multiplicamos los coeficientes de cada color por los vectores
    __m256 res1 = _mm256_mul_ps(vec1f, coeficiente);
    __m256 res2 = _mm256_mul_ps(vec2f, coeficiente);

    //Sumamos horizontalmente para obtener el vector final
    __m256 res = _mm256_hadd_ps(res1, res2);
    __m256 total = _mm256_hadd_ps(res, res);

    // ordenamos los valores
    total = _mm256_permutevar8x32_ps(total, orden);
    // Convertimos a entero de 32 bits
    __m128i gris = _mm_cvtps_epi32(_mm256_extractf128_ps(total, 0));

    // Guardamos los valores
    uint32_t *ptr = (__m128i *) &gris;

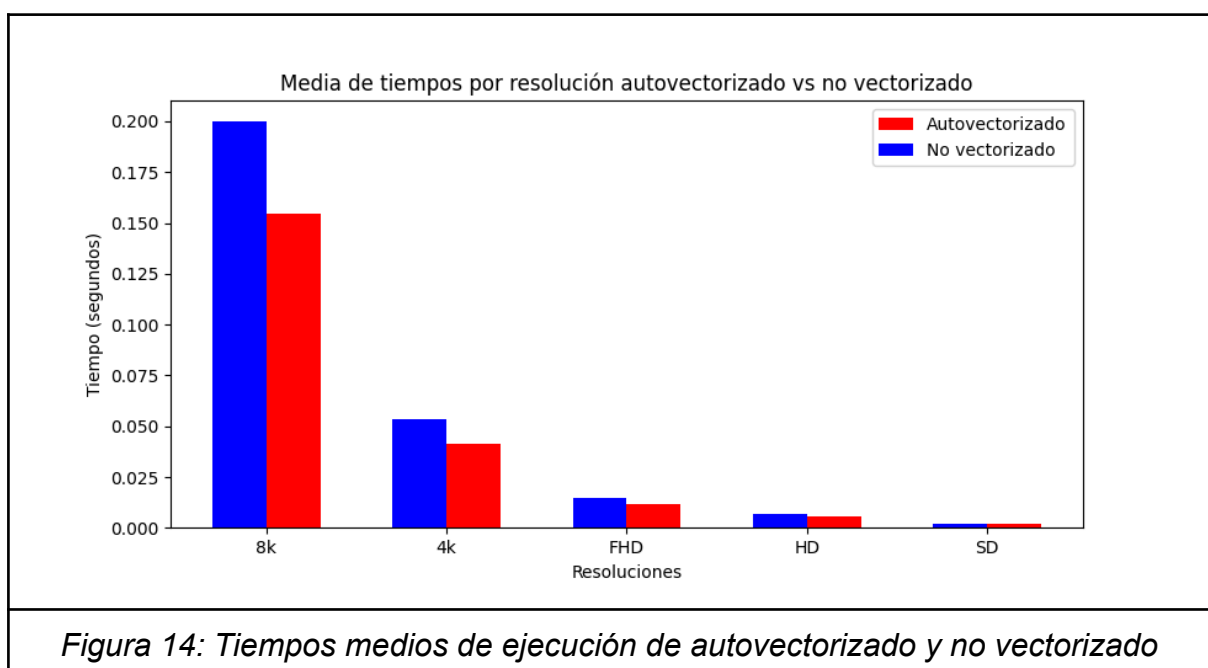
    grey_image[(j * width) + i] = ptr[0];
    grey_image[(j * width) + i + 1] = ptr[1];
    grey_image[(j * width) + i + 2] = ptr[2];
    grey_image[(j * width) + i + 3] = ptr[3];
}

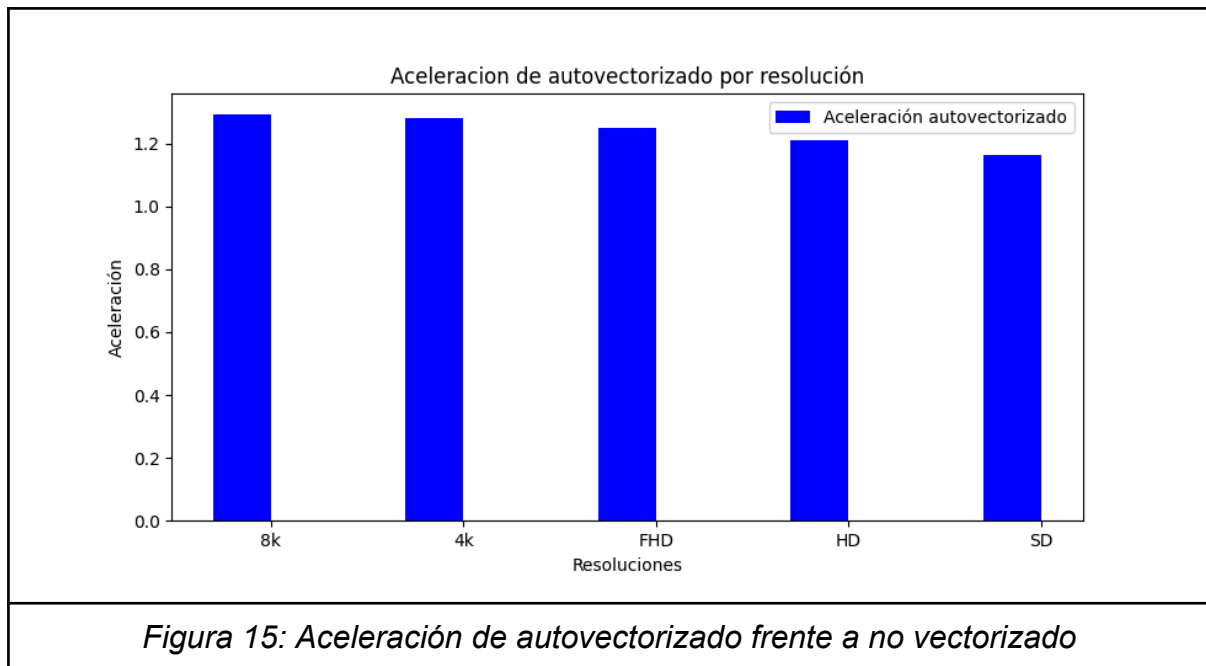
```

Figura 13: Código de greyScale.c con intrinsics

En primer lugar hemos creado dos vectores para poder guardar dos píxeles en cada vector ya que cada **pixel** ocupa **4 bytes**. Luego para poder operar con estos vectores tenemos que convertir los datos en **float** por lo que primero los pasamos a **enteros sin signo** para después pasarlos a **float**. A continuación multiplicamos los vectores ya convertidos por los **coeficientes** de cada color. Una vez hemos multiplicado hacemos una suma **horizontal** para obtener los valores de cada color de cada pixel y sumarlos todos juntos. Luego hemos ordenado todos los valores y convertido el resultado final en un **entero de 32 bits** para luego almacenarlos en la imagen.

Mejoras realizadas con autovectorización

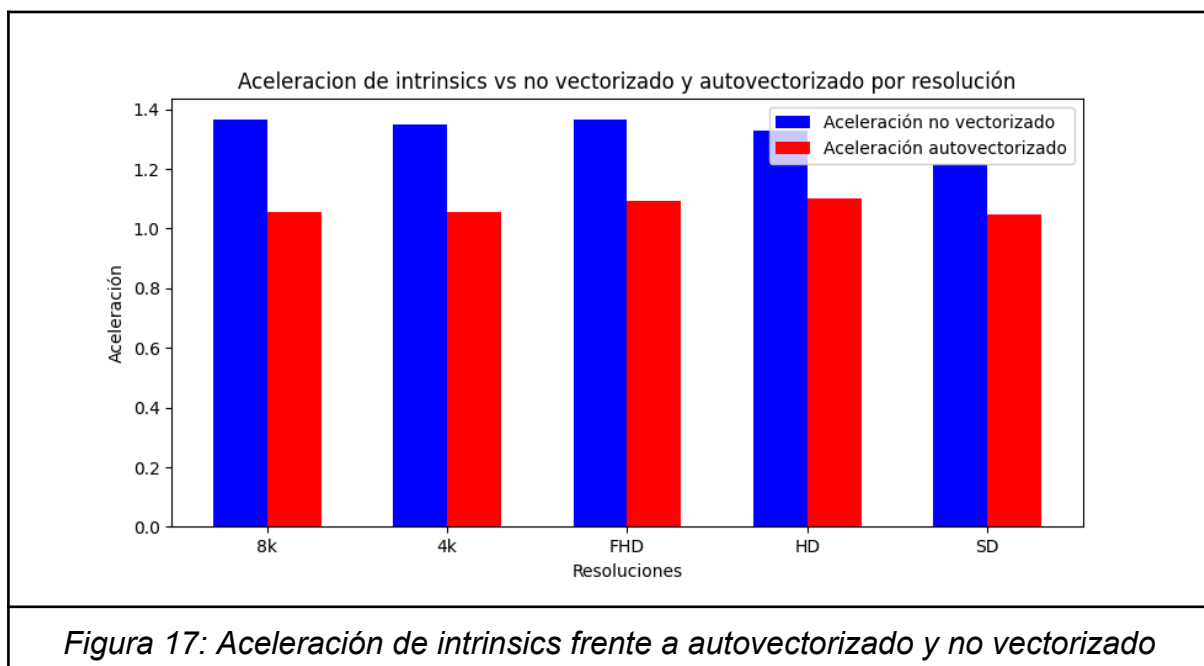
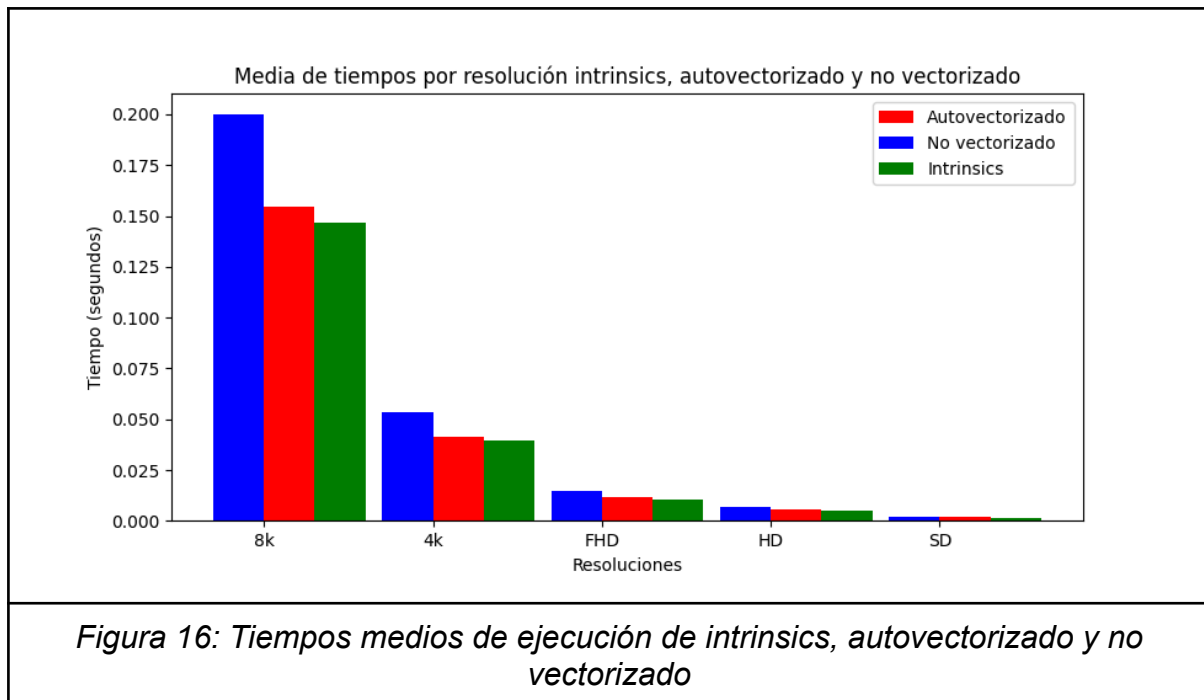




Se ha compilado el código no vectorizado empleando la bandera **-O3** pero con la opción **-fno-tree-vectorize** para obtener las mismas optimizaciones que al compilar solo con **-O3**, salvando la vectorización, de esta forma la comparación enfatiza la diferencia entre la vectorización y no vectorización.

Analizando los resultados la autovectorización resulta en una mejora de los tiempos de ejecución, teniendo una aceleración en torno al 1.2 que escala ligeramente a medida que aumenta el tamaño de la imagen, sin embargo, esta mejora es relativamente pequeña.

Mejoras realizadas con vectorización manual usando intrinsics



Los tiempos de ejecución de intrinsics están ligeramente por debajo de los del código autovectorizado, lo cual significa que se ha obtenido una mejora con respecto a la autovectorización y por lo tanto frente al código no vectorizado. Aunque con respecto a la autovectorización la mejora es mínima siendo la

aceleración es cercana 1, esto puede deberse a que los compiladores tienen un buen desempeño al vectorizar y paralelizar código.

Si no empleamos la bandera -O3 en la compilación del código no vectorizado tenemos unas diferencias más significativas dado que no se estarían aplicando ni la vectorización ni el resto de optimizaciones:

Imagen	Intrinsics	Autovectorizado	Sin vectorizar	Speedup Intrinsics	Speedup Autovectorizado
SD	0,002947	0,004508	0,019011	6,45097	4,21717
HD	0,015114	0,016176	0,053881	3,56497	3,33092
FHD	0,025545	0,026435	0,136919	5,35991	5,17946
4k	0,092932	0,103137	0,479613	5,16090	4,65025
8k	0,314202	0,341955	1,716752	5,46385	5,02040

